



ASISTENCIA INTELIGENTE AL APARCAMIENTO MEDIANTE APRENDIZAJE POR REFUERZO EN UN ENTORNO SIMULADO

APRENDIZAJE AUTOMÁTICO II

Laura Somolinos Sierra, Alejandro Blanco Castellanos

ÍNDICE

- 1) MOTIVACIÓN DEL PROBLEMA**
- 2) MODELADO DEL PROBLEMA (MDP)**
 - ESPACIO DE ESTADOS
 - ESPACIO DE ACCIONES
 - FUNCIÓN DE RECOMPENSA
 - DINÁMICA DE TRANSICIÓN
 - FACTOR DE DESCUENTO
- 3) ESTADO DEL ARTE**
- 4) SOLUCIONES PROPUESTAS Y DISEÑO EXPERIMENTAL**
 - Q-LEARNING
 - SARSA
- 5) SOLUCIONES CON DEEP REINFORCEMENT LEARNING**
 - DEEP Q-NETWORKS
 - DOUBLE DQN
 - DUELING DQN
- 6) CONCLUSIONES Y TRABAJO FUTURO**

1) MOTIVACIÓN DEL PROBLEMA:

Los sistemas de asistencia al aparcamiento y, en general, las maniobras de conducción autónoma en espacios reducidos constituyen un problema relevante dentro de la Inteligencia Artificial, la Ciencia de Datos y la robótica móvil. En este tipo de problemas, un agente debe tomar decisiones secuenciales a partir de la información que le proporciona el entorno (en nuestro caso información parcial), como medidas de distancia a obstáculos, posición relativa respecto a la plaza y orientación del vehículo. El agente debe tomar una secuencia de decisiones que afectan progresivamente al estado del entorno. Debe aprender no solo a acercarse a una posición objetivo, sino también a hacerlo evitando colisiones, manteniendo una orientación adecuada y decidiendo cuándo detenerse. Esto hace que el problema sea más complejo que una simple tarea de clasificación o regresión, ya que el resultado final depende de la calidad de toda la trayectoria seguida y no únicamente de una decisión aislada. Esta tarea resulta especialmente interesante desde la perspectiva de la Inteligencia Artificial porque combina percepción, control, planificación y toma de decisiones bajo restricciones de seguridad.

Desde el punto de vista de la Ciencia de Datos, el problema resulta relevante porque obliga a transformar información continua del entorno en variables útiles para el aprendizaje. En nuestro caso, el vehículo recibe observaciones como posición, orientación, velocidad, distancia al objetivo y lecturas de sensores simulados. Esta información debe ser procesada para que distintos algoritmos puedan utilizarla: los métodos tabulares, como Q-Learning y SARSA, requieren discretizar el estado, mientras que métodos como DQN pueden trabajar directamente con representaciones continuas mediante redes neuronales. Por tanto, el problema permite analizar cómo la representación de los datos influye directamente en el aprendizaje y en el comportamiento final del agente.

Tradicionalmente las maniobras de aparcamiento se han abordado mediante controladores basados en reglas, planificación geométrica o algoritmos clásicos de búsqueda. Sin embargo, el Aprendizaje por Refuerzo permite plantear el problema desde una perspectiva diferente: el agente no recibe una trayectoria óptima predefinida, sino que aprende mediante interacción con el propio entorno simulado, recibiendo recompensas o penalizaciones y así se da cuenta de que acciones le convienen más.

Los sistemas de asistencia al aparcamiento están presentes en vehículos modernos, robots móviles de almacén, plataformas autónomas de reparto, robots de inspección y sistemas de navegación en entornos cerrados. En todos estos casos, el agente debe moverse en espacios reducidos, evitar obstáculos y alcanzar una posición objetivo con una orientación determinada.

En este trabajo se propone un entorno simulado de aparcamiento, compatible con la API de Gymnasium, donde nuestro vehículo debe aprender a aparcar en una plaza en la otra esquina contraria a donde empieza evitando colisiones con obstáculos. Se ha decidido

simular como si hubiera otros vehículos por el camino y con una alta penalización por colisión se intenta que el objeto llegue a donde se espera, a la “plaza de aparcamiento” que se sitúa entre dos coches. También es una tarea adecuada para ser formulada como un Proceso de Decisión de Markov que es otro objetivo principal de la práctica

2) MODELADO DEL PROBLEMA: FORMALIZACION MDP

El problema de aparcamiento autónomo se modela como un proceso de Decisión de Markov definido por $MDP = (S, A, P, R, \gamma)$

S = espacio de estados

A = espacio de acciones

P = dinámica de transición

R = función de recompensa

Γ = factor de descuento

En este trabajo, el agente representa el controlador del vehículo y el entorno simulado representa una zona de aparcamiento bidimensional con obstáculos, sensores de distancia y una plaza objetivo.

En cada instante temporal t , el agente recibe una observación del entorno, selecciona una acción y el simulador actualiza la posición, orientación y velocidad del vehículo. El objetivo del agente es aprender una política $\pi(s)$ que maximice la recompensa acumulada esperada durante el episodio.

- Espacio de estados S :

El estado interno del simulador se define mediante variables continuas que describen la situación física del vehículo:

$$st = (xt, yt, \theta t, vt)$$

Siendo x la posición horizontal del vehículo, y la posición vertical, θ la orientación del vehículo en radianes y v la velocidad lineal actual.

El entorno se desarrolla en un mundo continuo de tamaño 10x6 metros. Sin embargo, el agente no recibe únicamente estas variables internas sino una observación continua que incluye información adicional útil para la toma de decisiones. La observación utilizada tiene dimensión 18:

- $x, y \rightarrow$ posición normalizada del vehículo
- $\cos(\theta), \sin(\theta) \rightarrow$ orientación del vehículo
- $v \rightarrow$ velocidad normalizada
- $d1, \dots, d8 \rightarrow$ normalizaciones de los valores que nos dan los sensores de distancia
- $dg \rightarrow$ distancia normalizada al centro de la plaza
- $\cos(\alpha), \sin(\alpha) \rightarrow$ dirección relativa hacia la plaza
- $\cos(e), \sin(e) \rightarrow$ error de orientación respecto a la orientación objetivo

Se utilizan seno y coseno para representar los ángulos porque sino habría discontinuidades con $-\pi$ y π .

El espacio de observaciones es por tanto continuo pero para poder aplicar algoritmos tabulares como Q-learning y SARSA se realiza una discretización de la

observación continua y esa discretización transforma el vector de 18 variables en tuplas discretas formadas por variables relevantes como: posición aproximada, distancia al objetivo, error angular, sensores principales y velocidad. Eso permite construir una tabla $Q(s,a)$ aunque introduce una pérdida de precisión respecto al estado continuo original.

- Espacio de acciones A:

Aunque en un vehículo real las acciones de control suelen ser continuas, como el ángulo de giro del volante o la aceleración, en este trabajo se define un espacio de acciones discreto. Esta decisión permite comparar de forma homogénea Q-Learning, SARSA y DQN sobre el mismo entorno.

ACCION	DESCRIPCION	VELOCIDAD	GIRO
0	Avanzar recto	+v max	0
1	Avanzar girando a la izquierda	+v max	+w max
2	Avanzar girando a la derecha	+v max	-w max
3	Retroceder recto	-v max	0
4	Retroceder girando a la izquierda	-v max	+w max
5	Retroceder girando a la derecha	-v max	-w max
6	Frenar/detenerse	0	0

Por lo tanto $A = \{0,1,2,3,4,5,6\}$

Este conjunto de acciones permite modelar una maniobra de aparcamiento simplificada, donde el vehículo debe combinar desplazamientos hacia delante, hacia atrás, giros y una acción final de frenado.

- Dinámica de transición P

Define cómo evoluciona el entorno tras aplicar una acción. En este trabajo se utiliza un modelo simplificado de vehículo en dos dimensiones. Cada acción se traduce en una velocidad lineal v_t y una velocidad angular ω_t . A partir de ellas, el entorno actualiza la posición y orientación del vehículo mediante:

$$\begin{aligned}
 x_{t+1} &= x_t + v_t \cos(\theta_t) \Delta t \\
 y_{t+1} &= y_t + v_t \sin(\theta_t) \Delta t \\
 \theta_{t+1} &= \theta_t + \omega_t \Delta t
 \end{aligned}$$

Donde $\Delta t = 0.1$ s es el paso temporal

En el entorno la transición es determinista ya que dado un estado s y una acción a el siguiente estado queda determinado por las ecuaciones anteriores. El episodio finaliza con una de estas acciones:

- El vehículo aparca correctamente
- El vehículo colisiona con un obstáculo o con los límites del entorno
- Se alcanza el número máximo de pasos

- El vehículo permanece demasiados pasos sin mejorar su distancia al objetivo (se bloquea)

Aunque la conducción autónoma en general puede verse como una tarea continua, en esta práctica el problema se formula como una tarea episódica. Cada episodio comienza con el vehículo en una posición inicial y termina cuando se cumple alguna de las condiciones anteriores.

- Función recompensa R:

En este problema la recompensa se ha diseñado a para favorecer una maniobra de aparcamiento segura y eficiente evitando comportamientos como colisionar, detenerse lejos de la plaza o quedarse oscilando hacia delante y hacia atrás.

La recompensa base incluye:

Penalización pequeña por paso lo cual incentiva a resolver la tarea en el menor numero de pasos posibles.

Se añade una recompensa proporcional al progreso hacia la plaza:

$R_t = r_t + \lambda (d(t-1) - d(t))$ siendo $d(t)$ la distancia del vehículo al centro de la plaza en el instante t . Si el agente se acerca al objetivo, el término es positivo; si se aleja, es negativo.

Se recompensa la mejora en la orientación : $R_t = r_t + \lambda (e(\theta, t-1) - e(\theta, t))$ Donde $e(\theta, t)$ representa el error angular entre la orientación del vehículo y la orientación objetivo de aparcamiento.

Otras recompensas positivas serían:

Acercarse al objetivo, mejorar la orientación, frenar cerca de la plaza y alineado, aparcar correctamente (con una recompensa muy alta)

Otras penalizaciones:

Estar demasiado cerca de obstáculos, colisionar (penalización fuerte), oscilar entre acciones opuestas, frenar fuera de la zona correcta

La condición de éxito absoluto sería:

$$dt < d_{max}, \quad |e_{\theta, t}| < e_{max}, \quad |v_t| < v_{maxstop}, \quad a_t = \text{frenar}$$

el vehículo debe estar suficientemente cerca del centro de la plaza, con una orientación adecuada, prácticamente detenido y habiendo ejecutado la acción de frenar.

Durante los primeros experimentos se observó que el agente podía aprender comportamientos cíclicos, alternando acciones opuestas sin completar la maniobra. Para evitar este comportamiento, se incorporó una penalización por oscilación entre acciones inversas y una condición de truncamiento por estancamiento.

- Factor de descuento γ

Determina la importancia relativa de las recompensas futuras frente a las inmediatas, en este trabajo se utiliza un valor alto: $\gamma=0.98$

Esta elección se justifica porque el aparcamiento requiere planificación a medio plazo. Algunas acciones pueden no producir una mejora inmediata, pero ser necesarias para orientar correctamente el vehículo y completar la maniobra más adelante. Por ejemplo, retroceder o girar puede aumentar temporalmente la

distancia al centro de la plaza, pero ser útil para alcanzar una posición final adecuada.

Un valor bajo de γ haría que el agente priorizara recompensas inmediatas, como acercarse rápidamente al objetivo, aunque eso pudiera conducir a una mala orientación o a una colisión. En cambio, un valor alto permite que el agente valore la recompensa futura asociada al aparcamiento correcto.

3) ESTADO DEL ARTE:

El problema abordado en este trabajo se relaciona con la navegación autónoma y la planificación de maniobras en entornos con obstáculos, concretamente con una tarea simplificada de aparcamiento autónomo. Este tipo de problemas ha sido estudiado tradicionalmente desde la robótica móvil y la conducción autónoma mediante técnicas de planificación de movimiento, control clásico y, más recientemente, Aprendizaje por Refuerzo.

En la industria, el aparcamiento y la navegación en espacios reducidos suelen abordarse mediante una arquitectura dividida en varias etapas: percepción del entorno, planificación de trayectoria y control del vehículo. La planificación puede resolverse con métodos como A*, Dijkstra, RRT o variantes de planificación geométrica, mientras que el seguimiento de la trayectoria suele realizarse con controladores clásicos, como PID, Pure Pursuit, Stanley Controller o control predictivo basado en modelo. En una revisión sobre planificación y control para vehículos autónomos urbanos, Paden et al en su obra *A Survey of Motion Planning and Control Techniques for Self-driving Urban Vehicles*, destacan que las tareas críticas de un vehículo autónomo incluyen la planificación de movimientos en entornos dinámicos y su ejecución robusta mediante control realimentado. También señalan que las distintas técnicas se diferencian por el modelo de vehículo utilizado, las hipótesis sobre el entorno y los requisitos computacionales.

Estos enfoques clásicos tienen la ventaja de ser interpretables, eficientes y ampliamente utilizados en sistemas reales. Sin embargo, suelen requerir un diseño explícito de la trayectoria o de las reglas de control. En el caso de una maniobra de aparcamiento, esto implica definir manualmente cómo debe aproximarse el vehículo, cuándo debe girar, cuándo debe retroceder y cuándo debe detenerse. Este diseño puede funcionar bien en escenarios controlados, pero puede ser menos flexible cuando el entorno cambia, cuando existen incertidumbres en los sensores o cuando la maniobra requiere adaptarse a distintas configuraciones de obstáculos.

El Aprendizaje por Refuerzo plantea una alternativa distinta: en lugar de programar explícitamente todas las reglas de decisión, se define una función de recompensa y el agente aprende una política mediante interacción con el entorno. Esta perspectiva resulta especialmente interesante en robótica, donde muchas tareas son difíciles de especificar manualmente, pero sí pueden evaluarse a través de recompensas o penalizaciones. Kober, Bagnell y Peters en su obra *Reinforcement Learning in Robotics: A Survey*, explican en su revisión sobre Aprendizaje por Refuerzo en robótica que RL proporciona un marco para que los robots descubran comportamientos óptimos mediante ensayo y error, aunque también remarcen desafíos importantes como la alta dimensionalidad, el coste de la exploración y la transferencia entre simulación y mundo real.

En problemas de navegación y aparcamiento, los métodos tabulares como Q-Learning o SARSA permiten aprender una función de valor $Q(s, a)$ cuando el espacio de estados y acciones es discreto o ha sido discretizado. Q-Learning es un método off-policy, porque actualiza el valor de una acción utilizando la mejor acción estimada en el estado siguiente. SARSA, en cambio, es on-policy, ya que actualiza usando la acción que realmente tomará la política actual. Esta diferencia puede ser relevante en entornos con riesgo de colisión: SARSA puede aprender políticas más conservadoras porque tiene en cuenta la exploración real del agente, mientras que Q-Learning tiende a aprender una política más optimista basada en el máximo valor futuro estimado.

No obstante, los métodos tabulares tienen una limitación clara: no escalan bien cuando el estado es continuo o de alta dimensión. En nuestro caso, el entorno contiene variables continuas como posición, orientación, velocidad, distancia al objetivo y sensores simulados. Para aplicar Q-Learning y SARSA es necesario discretizar esta observación, lo que reduce la precisión de la política aprendida y puede producir pérdida de información. Esta limitación motiva el uso de métodos basados en Deep Learning, como Deep Q-Networks.

DQN fue popularizado por Mnih et al. *Human-level control through deep reinforcement learning*, quienes demostraron que una red neuronal podía aproximar la función $Q(s, a)$ y aprender políticas directamente a partir de entradas de alta dimensión. Su trabajo supuso un avance importante en Deep Reinforcement Learning, ya que combinaba Q-Learning con redes neuronales, experience replay y una red objetivo para estabilizar el entrenamiento. Aunque su dominio experimental eran videojuegos de Atari, la idea principal es aplicable a otros problemas donde el espacio de estados es demasiado grande o continuo para una tabla Q tradicional.

En tareas de control continuo, como conducción, robótica o aparcamiento, también se han desarrollado métodos de Deep Reinforcement Learning más específicos. Lillicrap et al. (*Continuous control with deep reinforcement learning*) propusieron DDPG, un algoritmo actor-critic capaz de trabajar con espacios de acciones continuos, extendiendo ideas de DQN a problemas de control físico. Este tipo de algoritmos es especialmente relevante para vehículos reales, donde acciones como aceleración o ángulo de giro son continuas. Sin embargo, en este trabajo se opta por discretizar el espacio de acciones para poder comparar de forma homogénea Q-Learning, SARSA y DQN sobre el mismo entorno.

También existen trabajos más cercanos al aparcamiento autónomo. Por ejemplo, algunos enfoques recientes han aplicado Deep Reinforcement Learning a tareas de aparcamiento valet autónomo o aparcamiento en simuladores de alta fidelidad, combinando planificación, percepción y políticas aprendidas. Khalid et al. plantean un marco de autonomous parking en ciudades inteligentes y proponen, entre otros métodos, un enfoque basado en DQN para tomar decisiones en un entorno dinámico. Estos trabajos muestran que el aparcamiento autónomo puede formularse como un problema secuencial de decisión, donde el agente debe optimizar trayectorias, evitar obstáculos y cumplir restricciones del entorno.

4) Soluciones propuestas y diseño experimental

4.1) Enfoque general

Para resolver el problema de aparcamiento autónomo se han probado distintos enfoques basados en Aprendizaje por Refuerzo. El objetivo era entrenar un agente capaz de controlar un vehículo en un entorno continuo simulado, evitando obstáculos y finalizando la maniobra dentro de la plaza de aparcamiento.

El entorno implementado devuelve una observación continua de 18 variables, compuesta por información de posición, orientación, velocidad, sensores de distancia, distancia al objetivo y error angular. Sin embargo, los algoritmos tabulares utilizados, Q-Learning y SARSA, requieren un espacio de estados discreto. Por ello, se introdujo una etapa de discretización de la observación, transformando las variables continuas más relevantes en una tupla discreta que pudiera utilizarse como clave de la tabla Q.

Se han considerado dos enfoques experimentales para los algoritmos discretos:

1. **Q-Learning v3**: versión final de Q-Learning, entrenada tras ajustar la función de recompensa y añadir penalizaciones por riesgo, oscilación y estancamiento.
2. **SARSA**: algoritmo on-policy entrenado sobre el mismo entorno final que Q-Learning v3, para comparar su comportamiento frente a un método off-policy.

4.2) Entorno experimental

El entorno desarrollado se implementa siguiendo la API de Gymnasium. El vehículo se mueve en un mundo continuo de tamaño 10×6 metros, con una plaza de aparcamiento situada cerca del punto (8.5, 3.0) y varios obstáculos rectangulares que simulan vehículos aparcados y obstáculos laterales. El agente dispone de 7 acciones discretas: avanzar recto, avanzar girando a izquierda o derecha, retroceder recto, retroceder girando a izquierda o derecha, y frenar.

La observación del entorno contiene posición normalizada, orientación representada mediante seno y coseno, velocidad, ocho sensores de distancia, distancia al objetivo, dirección relativa hacia la plaza y error de orientación. Esta representación permite conservar información continua del entorno, aunque para Q-Learning y SARSA se transforma posteriormente en estados discretos. El entorno también incorpora una función de renderizado para visualizar la política aprendida sobre el mapa de aparcamiento.

La función de recompensa se diseñó de forma incremental. Inicialmente, el agente aprendía a aproximarse a la plaza, pero podía quedarse oscilando hacia delante y hacia atrás o detenerse en posiciones poco útiles. Por ello, en la versión final se introdujeron varios términos:

Componente de recompensa	Objetivo
Penalización por paso	Favorecer trayectorias más cortas

Componente de recompensa	Objetivo
Recompensa por acercarse al objetivo	Guiar al agente hacia la plaza
Penalización por falta de progreso	Evitar movimientos sin avance real
Recompensa por mejorar orientación	Favorecer una entrada alineada
Penalización por riesgo	Evitar acercarse demasiado a obstáculos
Penalización por oscilación	Reducir ciclos de avance-retroceso
Penalización por colisión	Evitar políticas inseguras
Bonus por centrado	Incentivar que el vehículo se acerque al centro de la plaza
Recompensa por aparcar	Premiar el estado terminal correcto

Además, se añadió una condición de truncamiento por estancamiento. Si durante demasiados pasos el agente no mejora su mejor distancia al objetivo, el episodio se corta y recibe una penalización. Esto permitió evitar que el agente permaneciera indefinidamente en ciclos de acciones sin resolver la tarea.

4.3) Q-Learning

El primer algoritmo implementado fue **Q-Learning**, un método tabular basado en valores. El objetivo de Q-Learning es aprender una función $Q(s, a)$, que estima la recompensa acumulada esperada al tomar una acción a en un estado s y seguir posteriormente la mejor política aprendida.

Q-Learning es un algoritmo **off-policy**, ya que actualiza la tabla Q suponiendo que en el siguiente estado se tomará la mejor acción posible, aunque durante el entrenamiento el agente explore con una política ϵ -greedy. Su regla de actualización es:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

donde α es la tasa de aprendizaje, γ el factor de descuento, r la recompensa recibida, s' el siguiente estado y $\max_{a'} Q(s', a')$ el valor de la mejor acción futura estimada.

En este problema, Q-Learning tiende a aprender una política orientada al objetivo final, ya que siempre actualiza usando la mejor acción estimada en el siguiente estado. Esto puede hacer que, una vez encuentra una trayectoria eficaz para entrar en la plaza, la política final sea bastante limpia. Sin embargo, al trabajar sobre una discretización del estado continuo, también puede aprender trayectorias no perfectamente suaves o realizar maniobras previas poco directas.

Se realizaron dos entrenamientos principales con Q-Learning:

Q-Learning v2

Esta fue la primera versión funcional. Obtuvo buenos resultados agregados, con una tasa de éxito alta y recompensa media elevada. Sin embargo, fue entrenada con una

versión anterior del entorno, antes de endurecer la función de recompensa. Por tanto, aunque las métricas son buenas, no se considera la versión final más representativa.

Q-Learning v3

Esta versión fue entrenada después de modificar el entorno para penalizar mejor el riesgo, la oscilación y el estancamiento. Aunque las métricas medias son inferiores a Q-Learning v2, representa mejor el comportamiento del agente en el entorno definitivo.

En la evaluación visual de la política greedy, Q-Learning v3 consigue aparcar correctamente sin colisionar. El agente no toma la trayectoria más directa hacia la plaza, ya que realiza varias maniobras previas, pero una vez alcanza la entrada de la zona de aparcamiento, entra y frena correctamente. En la ejecución visual guardada, completa la maniobra aproximadamente en el paso 244, con éxito y sin colisión.

2.4.) SARSA

El segundo algoritmo implementado fue **SARSA** (*State-Action-Reward-State-Action*). Al igual que Q-Learning, SARSA aprende una función $Q(s, a)$, pero la actualización se realiza de forma distinta.

SARSA es un método **on-policy**, lo que significa que aprende teniendo en cuenta la acción que realmente selecciona la política actual. Su regla de actualización es:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)]$$

donde a' es la siguiente acción seleccionada por la política ϵ -greedy.

La diferencia principal respecto a Q-Learning es que SARSA no actualiza suponiendo la mejor acción posible, sino la acción real que el agente tomará durante el entrenamiento. Esto puede producir políticas más sensibles a la exploración y, en algunos casos, más conservadoras o con más maniobras de ajuste.

En este entorno concreto, SARSA mostró un comportamiento interesante: **se dirige de forma más directa hacia la plaza de aparcamiento que Q-Learning**, pero al llegar a la zona de entrada tiende a quedarse más tiempo ajustando la posición mediante movimientos de avance y retroceso. Esto hace que tarde más pasos en completar la maniobra, aunque también consigue aparcar correctamente sin colisionar.

En la evaluación visual, SARSA completa la tarea aproximadamente en el paso 284. Su recompensa total en esa ejecución es mayor que la de Q-Learning, probablemente porque durante buena parte de la trayectoria se aproxima más directamente al objetivo. Sin embargo, necesita más pasos porque se queda ajustando la maniobra final antes de frenar.

4.5) Hiperparámetros

Los hiperparámetros utilizados para los algoritmos tabulares fueron los siguientes:

Hiperparámetro	Q-Learning v3	SARSA	Significado
Episodios	7000	7000	Número total de episodios de entrenamiento
Máximo de pasos	300	300	Límite de pasos por episodio
α	0.08	0.06	Tasa de aprendizaje
γ	0.98	0.98	Factor de descuento
$\epsilon_{\text{inicial}}$	1.0	1.0	Exploración inicial
ϵ_{min}	0.02	0.02	Exploración mínima
Decaimiento de ϵ	0.995	0.999	Reducción progresiva de la exploración
Política de exploración	ϵ -greedy	ϵ -greedy	Selección de acciones

La tasa de aprendizaje α controla cuánto se actualiza la tabla Q en cada transición. Para Q-Learning se utilizó $\alpha = 0.08$, mientras que para SARSA se redujo a $\alpha = 0.06$. Esta reducción se hizo porque SARSA es más sensible a la acción realmente tomada bajo exploración, por lo que una tasa algo menor ayuda a estabilizar el aprendizaje.

El factor de descuento $\gamma = 0.98$ da bastante peso a las recompensas futuras. Esto es adecuado en una maniobra de aparcamiento, ya que algunas acciones pueden no ser beneficiosas de forma inmediata pero sí necesarias para lograr una orientación correcta más adelante.

La política de exploración utilizada fue ϵ -greedy. Al inicio, $\epsilon = 1.0$, por lo que el agente explora de forma prácticamente aleatoria. A medida que avanza el entrenamiento, ϵ disminuye hasta un mínimo de 0.02, permitiendo que el agente explore la política aprendida sin eliminar completamente la exploración.

En SARSA se utilizó un decaimiento más lento de ϵ , concretamente 0.999, porque con un decaimiento demasiado rápido el agente podía fijar prematuramente políticas subóptimas. Mantener la exploración durante más episodios permitió que SARSA siguiera probando maniobras alternativas durante más tiempo.

4.6) Resultados experimentales

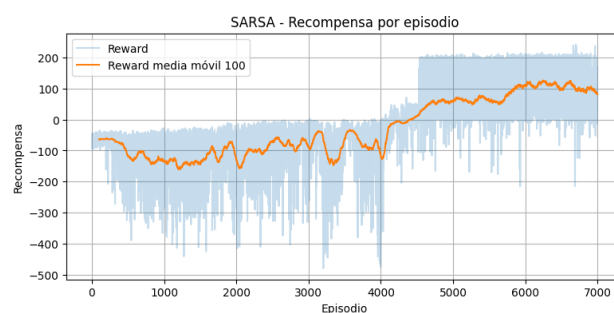
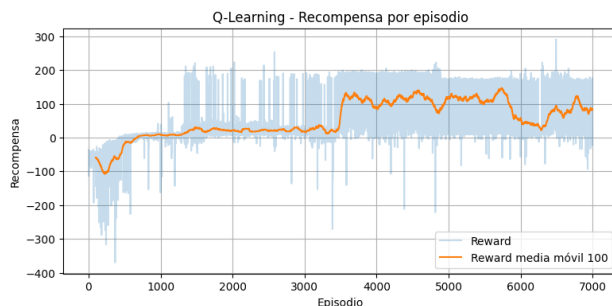
Para evaluar los algoritmos se registraron las siguientes métricas por episodio:

Métrica	Descripción
Recompensa acumulada	Suma de recompensas del episodio
Éxito	Indica si el vehículo aparcó correctamente
Colisión	Indica si el vehículo chocó
Pasos	Número de pasos hasta terminar el episodio
ϵ	Valor de exploración

Métrica	Descripción
Distancia final	Distancia final al centro de la plaza

En pruebas.ipynb se representan estas métricas mediante medias móviles para analizar la evolución del aprendizaje. Las gráficas principales son:

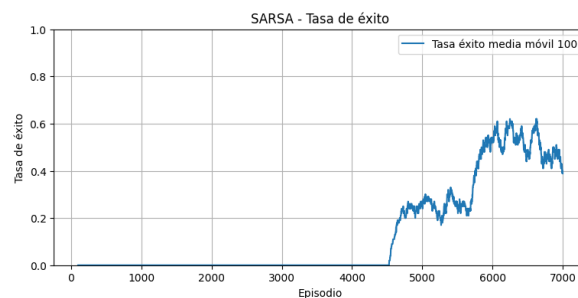
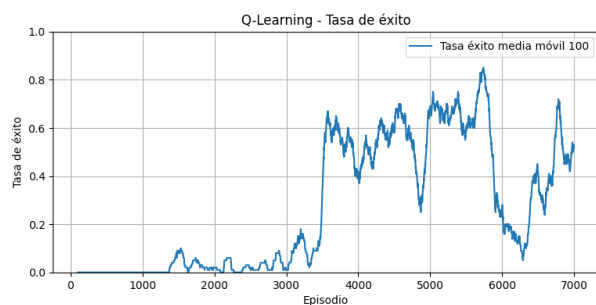
Evolución de la recompensa



La recompensa acumulada permite observar si el agente aprende políticas más útiles a lo largo del entrenamiento. En las primeras fases, la recompensa suele ser baja o inestable porque el agente explora mucho y realiza acciones poco eficientes. Conforme se actualiza la tabla Q, la recompensa media tiende a mejorar.

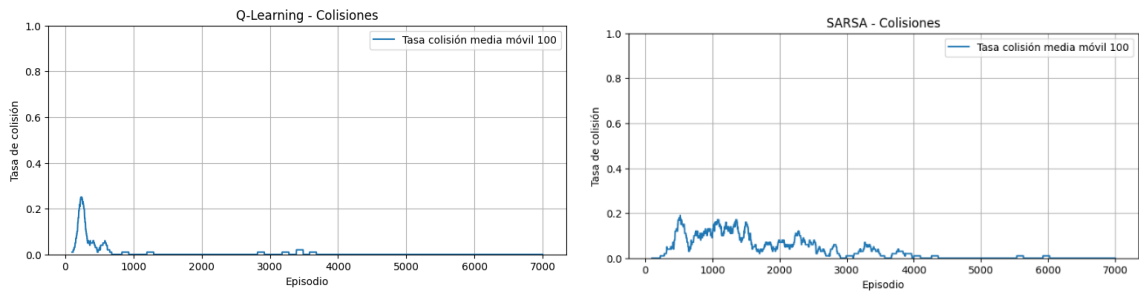
En estos dos casos, SARSA obtiene una recompensa media ligeramente superior en los últimos episodios, lo que concuerda con la observación visual de que se dirige de forma más directa hacia la plaza.

Tasa de éxito



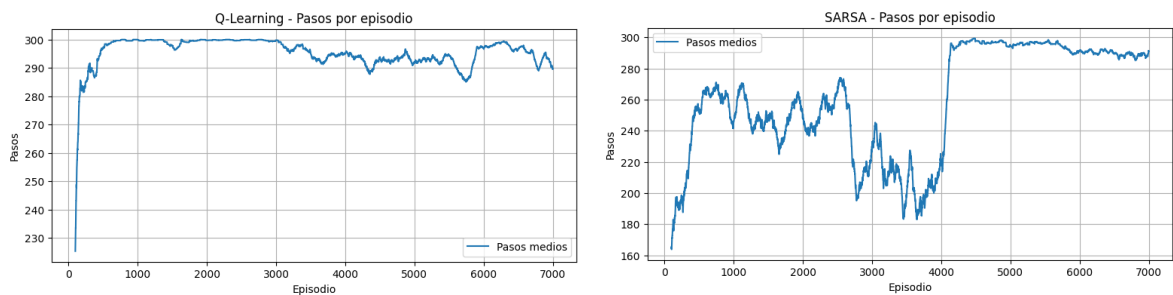
La tasa de éxito mide la proporción de episodios en los que el agente aparca correctamente. En el entorno final, Q-Learning v3 y SARSA obtienen tasas similares. Esto indica que ambos algoritmos son capaces de aprender una política válida, aunque no perfectamente estable. La dificultad se debe a que el entorno es continuo y los algoritmos tabulares dependen de una discretización del estado, lo que limita la precisión de la política.

Tasa de colisión



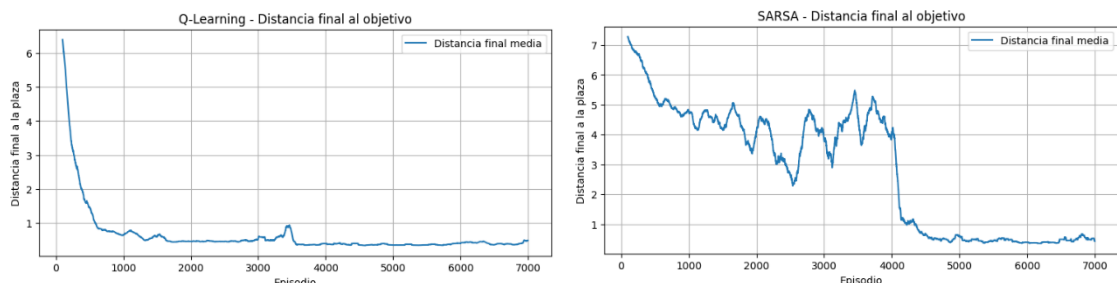
Una observación importante es que la tasa de colisión se mantiene prácticamente en cero en los resultados finales. Esto indica que la penalización por colisión y por proximidad a obstáculos cumple su objetivo y los agentes aprenden a evitar comportamientos inseguros. La ausencia de colisiones es especialmente relevante porque el problema no consiste solo en llegar a la plaza, sino en hacerlo de forma segura.

Pasos medios



El número de pasos permite evaluar la eficiencia de la maniobra. Q-Learning v3 necesita más pasos de media que Q-Learning v2, debido al entorno más estricto. SARSA reduce ligeramente los pasos medios respecto a Q-Learning v3 en las métricas agregadas, pero en la visualización final se observa que tarda más en completar la maniobra que Q-Learning, porque realiza ajustes cerca de la entrada de la plaza. Esta diferencia entre métrica agregada y visualización concreta es normal, ya que la tabla resume muchos episodios de entrenamiento, mientras que la visualización muestra una única ejecución greedy de la política final.

Distancia final al objetivo



La distancia final mide la precisión con la que el agente termina respecto al centro de la plaza podemos observar que ambos aprenden correctamente ya que esta distancia va disminuyendo con el paso de los episodios, aprende a quedarse cada vez más cerca. En la ejecución visual, ambos agentes terminan dentro del umbral permitido de aparcamiento, pero no exactamente en el centro. Esto se debe a que la condición de

éxito no exige distancia cero al centro, sino estar dentro de una tolerancia espacial y angular. Esta decisión hace viable el aprendizaje con métodos tabulares, pero limita la precisión final.

4.7) Comparación cuantitativa

En los últimos 500 episodios se obtuvieron los siguientes resultados medios:

	algoritmo	reward_medio	success_rate	collision_rate	steps_medio	final_distance_media
0	Q-Learning v2	181.440798	0.948	0.0	253.174	0.350782
1	Q-Learning v3	84.241179	0.462	0.0	293.368	0.407720
2	SARSA	98.550934	0.474	0.0	288.868	0.504246

Q-Learning v2 obtiene las mejores métricas agregadas, pero fue entrenado en una versión anterior del entorno. Por tanto, se considera un experimento preliminar útil para mostrar la evolución del diseño, pero no la versión final del sistema.

La comparación principal debe hacerse entre Q-Learning v3 y SARSA. Ambos algoritmos consiguen aparcar sin colisiones en la evaluación visual. SARSA obtiene una recompensa media y tasa de éxito ligeramente superiores en los últimos episodios, pero su distancia final media es peor. Además, visualmente se observa que SARSA va más directo hacia la plaza, pero se queda más tiempo ajustando la entrada mediante movimientos de avance y retroceso. Q-Learning v3 realiza una trayectoria menos directa, pero termina la maniobra de forma más limpia una vez entra en la zona de aparcamiento.

4.8) Interpretación de los comportamientos

La diferencia entre ambos algoritmos puede explicarse por su forma de actualización.

Q-Learning, al ser off-policy, aprende con la mejor acción futura estimada. Esto favorece políticas orientadas al objetivo final, aunque durante el entrenamiento se explore con acciones aleatorias. En este entorno, esto parece traducirse en una política que no siempre se aproxima por el camino más directo, pero que una vez encuentra la entrada adecuada de la plaza ejecuta una maniobra final más decidida.

SARSA, al ser on-policy, aprende usando la acción realmente seleccionada por la política exploratoria. Esto hace que sea más sensible a las acciones tomadas durante el proceso de exploración. En la práctica, SARSA aprende a acercarse más directamente a la plaza, pero cerca del objetivo realiza más correcciones, posiblemente porque la política ha aprendido a actuar con más cuidado ante pequeñas diferencias de orientación y posición.

En ambos casos, las políticas no son óptimas en sentido estricto. El problema original es continuo, pero ambos algoritmos utilizan una discretización del estado. Esto introduce pérdida de información y puede producir movimientos poco realistas, oscilaciones o aparcamientos que no quedan exactamente centrados. Aun así, los resultados muestran que los dos métodos son capaces de aprender una política válida para resolver la tarea.

5. SOLUCIONES CON DEEP LEARNING

5.1) DEEP Q-NETWORK (DQN)

El DQN constituye la primera aproximación de Deep Reinforcement Learning en este proyecto. Su idea central es reemplazar la Q-table por una red neuronal que aproxima la función de valor acción $Q(s, a; \theta)$, donde θ representa los pesos de la red. Esta sustitución permite trabajar directamente con el espacio de observación continuo de 18 variables, eliminando la necesidad de discretización.

En el DQN estándar se usa la misma red tanto para seleccionar la acción como para evaluar su valor. La ecuación que se minimiza en cada paso de entrenamiento es:

$$y = r + \gamma \cdot \max_{a'} Q(s', a'; \theta)$$

La función de pérdida es el error cuadrático medio entre el target y la predicción de la red:

$$L(\theta) = (r + \gamma \cdot \max_{a'} Q(s', a'; \theta) - Q(s, a; \theta))^2$$

La arquitectura de la red es completamente conectada con dos capas ocultas de 64 neuronas y activación ReLU, sin activación en la capa de salida para no acotar los Q-values:

- Entrada: vector de observación continuo de dimensión 18
- Capa oculta 1: Linear(18 → 64) + ReLU
- Capa oculta 2: Linear(64 → 64) + ReLU
- Salida: Linear(64 → 7), un Q-value por acción, sin activación

Dos mecanismos son fundamentales para estabilizar el entrenamiento del DQN. El primero es el experience replay: en lugar de entrenar con transiciones en orden cronológico, que introducirían correlaciones temporales dañinas, las transiciones se almacenan en un replay buffer de capacidad fija (60 000 en esta implementación) y se muestrea un mini-batch aleatorio en cada actualización. El segundo es el warmup: antes de empezar a aprender, el buffer se rellena con 5 000 transiciones generadas por una política completamente aleatoria, garantizando diversidad desde el primer batch.

La política de exploración es ϵ -greedy: con probabilidad ϵ se elige una acción aleatoria y con probabilidad $1 - \epsilon$ se elige la acción de mayor Q-value. ϵ decae multiplicativamente en cada episodio hasta alcanzar un mínimo de 0.1, asegurando exploración residual permanente.

- HIPERPARÁMETROS:

La siguiente tabla recoge los hiperparámetros con los que se obtuvo el mejor resultado en el entrenamiento del DQN, junto con la justificación de cada elección:

Hiperparámetro	Valor	Justificación
Episodios	7 000	Suficientes para superar la fase de exploración y estabilizar la política. El DQN necesita más episodios que los métodos más avanzados porque aprende más lentamente al no separar selección y evaluación de acciones.

Pasos máximos por episodio	300	Permite completar una maniobra de aparcamiento completa sin truncar episodios prematuramente.
Warmup steps	5 000	Rellena el buffer con experiencias aleatorias diversas antes del primer aprendizaje, evitando que los primeros batches estén sesgados.
Factor de descuento γ	0.99	Valor alto que prioriza la recompensa futura. Necesario porque aparcar requiere planificación: algunas acciones (retroceder, girar) son localmente malas pero globalmente necesarias.
Tasa de aprendizaje lr	5×10^{-4}	Valor moderado para Adam. Suficientemente bajo para que el entrenamiento sea estable y suficientemente alto para que la red converja en 7 000 episodios.
Tamaño de batch	128	Balance entre varianza del gradiente y coste computacional. Batches grandes reducen la varianza de cada actualización.
Tamaño del replay buffer	60 000	Permite almacenar 200 episodios completos. Lo suficientemente grande para romper correlaciones temporales sin comprometer la memoria disponible.
ϵ inicial	1.0	El agente empieza explorando completamente al azar para generar experiencias diversas desde el inicio.
ϵ mínimo	0.1	Se mantiene un 10 % de exploración residual durante todo el entrenamiento para evitar quedar atrapado en óptimos locales.
ϵ decay	0.999	Decaimiento lento: ϵ tarda 2 300 episodios en pasar de 1.0 a 0.1, dando tiempo al agente para explorar el entorno antes de explotar.
Frecuencia de aprendizaje	Cada 4 pasos	Reducir la frecuencia de llamadas a <code>learn()</code> abarata el coste computacional y permite incorporar nuevas experiencias al buffer entre actualizaciones.
Umbral de atasco	20 pasos / 0.05 m	Si el agente no avanza más de 0.05 m en 20 pasos consecutivos, el episodio se termina forzosamente

		para no llenar el buffer con transiciones poco informativas.
--	--	--

MEJORAS RESPECTO MÉTODOS TABULARES:

El DQN supone una mejora respecto a Q-Learning y SARSA en tres aspectos clave:

- Eliminación de la discretización: Q-Learning y SARSA requieren convertir el vector de 18 observaciones continuas en tuplas discretas, perdiendo precisión y resolución espacial. El DQN trabaja directamente con el vector continuo, lo que le permite distinguir estados que la discretización colapsaría en uno mismo.
- Generalización a estados no visitados: la red neuronal interpola entre estados vistos durante el entrenamiento, de modo que puede actuar razonablemente ante configuraciones del entorno que no ha encontrado exactamente. Una Q-table no tiene esta propiedad.
- Escalabilidad: el tamaño de la Q-table crece exponencialmente con la granularidad de la discretización. La red neuronal tiene un número fijo de parámetros independientemente de la resolución del espacio de estados.

Sin embargo, el DQN hereda una limitación importante: la misma red selecciona la acción de mayor Q-value y evalúa su valor para construir la ecuación de Bellman. Esto produce un sesgo positivo sistemático conocido como sobreestimación de Q-values, que puede llevar a políticas subóptimas. Esta limitación motiva directamente la variante Double DQN.

- RESULTADOS

El DQN fue entrenado durante 7 000 episodios. Los datos de entrenamiento revelan una convergencia real y significativa, aunque más lenta que los algoritmos posteriores. Hasta el episodio ~4 500 el agente acumula experiencia sin éxitos consistentes, y en torno a ese punto se produce un salto brusco: el porcentaje de episodios que terminan con el vehículo a menos de 0.35 m de la plaza pasa del 10 % al 80 %, manteniéndose en esa banda hasta el final del entrenamiento.

Esta distinción es importante: la condición de éxito estricta exige simultáneamente distancia < 0.35 m, error de orientación < 0.25 rad, velocidad prácticamente nula y acción de frenado. Que el agente cumpla esa conjunción exacta solo ocurre en 58 episodios de los últimos 1 000, pero que termine consistentemente dentro del radio de la plaza en el 80 % de los casos es una señal clara de convergencia real. El agente ha aprendido a llegar a la plaza; la dificultad reside en ejecutar el frenado preciso en el momento y orientación correctos.

En la evaluación visual greedy, el modelo guardado consigue aparcar con una distancia final al centro de la plaza de 0.063 m, completando la maniobra en 247 pasos.

- CONCLUSIONES

El DQN demuestra ser capaz de resolver la tarea de aparcamiento autónomo en el entorno continuo, superando claramente las limitaciones de precisión de los métodos tabulares. Su convergencia es real —el agente aprende a posicionarse dentro del radio de la plaza en más del 80 % de los episodios finales— pero es la más lenta de los tres algoritmos implementados, necesitando ~4 500 episodios para estabilizarse.

La principal limitación identificada es el sesgo de sobreestimación: al usar la misma red para seleccionar y evaluar acciones en el target de Bellman, el agente puede aprender valores Q inflados que ralentizan la convergencia y dificultan el cumplimiento simultáneo de todas las condiciones de aparcamiento. Esta limitación es la motivación directa para la implementación del Double DQN, que introduce una segunda red para desacoplar estas dos responsabilidades.

5.2) Double Deep Q-Network

El Double DQN parte de la misma base que el DQN estándar: una red neuronal entrenada con experience replay, warmup y política ϵ -greedy. La única diferencia es algorítmica y se concentra exclusivamente en cómo se construye el target de Bellman.

En el DQN estándar, la misma red selecciona la acción óptima y evalúa su valor, lo que introduce un sesgo positivo sistemático conocido como sobreestimación de Q-values:

$$y = r + \gamma \cdot \max_{a'} Q(s', a'; \theta) \quad [\text{DQN} \rightarrow \text{red única}]$$

El Double DQN corrige este problema separando dos responsabilidades en dos redes distintas:

- Red online (pesos θ): selecciona la acción de mayor Q-value en el estado siguiente.
- Red target (pesos θ^-): evalúa el valor de esa acción. Sus pesos permanecen congelados y se sincronizan con la online cada 1 000 pasos de aprendizaje.

El target de Bellman pasa a ser:

$$y = r + \gamma \cdot Q(s', \arg\max_{a'} Q(s', a'; \theta); \theta^-) \quad [\text{Double DQN} \rightarrow \text{redes desacopladas}]$$

Al desacoplar selección y evaluación, los errores de sobreestimación de la red online no se propagan directamente en el target, lo que estabiliza el entrenamiento y acelera la convergencia. La función de pérdida (MSE), el optimizador (Adam) y el resto del bucle de entrenamiento son idénticos al DQN.

La red tiene dos capas ocultas de 128 neuronas con ReLU, ligeramente más ancha que el DQN (64 neuronas), para capturar las relaciones más sutiles que requiere coordinar dos redes durante el aprendizaje.

HIPERPARÁMETROS SELECCIONADOS

La tabla recoge los hiperparámetros del Double DQN. Los que difieren del DQN se indican explícitamente en la justificación:

Hiperparámetros	Valor	Justificación
Episodios	5 000	La convergencia real ocurre en torno al episodio 2 600 — casi 2 000 antes que el DQN —, por lo que 5 000 episodios son suficientes para estabilizar la política.
Pasos máximos / episodio	300	Igual que el DQN; el horizonte temporal del problema no cambia.
Warmup steps	5 000	Igual que el DQN.

Factor de descuento γ	0.99	Igual que el DQN; el horizonte de planificación requerido es el mismo.
Tasa de aprendizaje l_r	5×10^{-4}	Igual que el DQN.
Tamaño de batch	128	Igual que el DQN.
Tamaño replay buffer	60 000	Igual que el DQN.
ϵ inicial	1.0	Igual que el DQN.
ϵ mínimo	0.1	Igual que el DQN.
ϵ decay	0.998	Ligeramente más rápido que en el DQN (0.999). Con menos episodios, conviene pasar antes a explotación para aprovechar la política más estable que proporciona la red target.
Sincronización red target	Cada 1 000 learn-steps	Los pesos de la red target se copian desde la online cada 1 000 pasos de aprendizaje. Un intervalo corto haría que ambas redes fueran casi idénticas (perdiendo el beneficio de la separación); uno largo introduciría targets obsoletos.
Frecuencia de aprendizaje	Cada 4 pasos	Igual que el DQN.
Arquitectura	18 $\rightarrow$ 128 $\rightarrow$ 128 $\rightarrow$ 7	Más ancha que el DQN (64 $\rightarrow$ 128 neuronas por capa) para capturar las relaciones más sutiles derivadas de coordinar dos redes.

- RESULTADOS

El Double DQN fue entrenado durante 5 000 episodios. La convergencia real, medida como el punto en que más del 70 % de los episodios terminan con el vehículo a menos de 0.35 m de la plaza, se produce en el episodio 2 664 aproximadamente 2 000 episodios antes que el DQN. A partir de ahí la política continúa mejorando: en los tramos finales el 89-97 % de los episodios terminan dentro del radio de la plaza, con una distancia media de 0.19 m en el mejor tramo (ep. 4 000-4 500).

Un patrón llamativo en los datos es que la tasa de éxito estricta no sigue una trayectoria monótona: alcanza un máximo local del 31 % en torno al episodio 3 000-3 250 y luego desciende, incluso mientras la distancia final media sigue bajando. Este comportamiento es coherente y no indica regresión: el agente está aprendiendo a posicionarse cada vez más cerca de la plaza (la distancia media cae de 1.9 m a 0.15 m entre los episodios 3 000 y 4 500), pero la condición de éxito exige cumplir simultáneamente distancia, orientación, velocidad y acción de

frenado. Alcanzar esas cuatro condiciones a la vez requiere una precisión que la exploración residual dificulta en los episodios intermedios.

En los últimos 500 episodios el reward medio es de 107.81, la tasa de colisiones cae a 0.002 y la distancia final media se sitúa en 0.27 m. En la evaluación greedy, el modelo consigue aparcar con una distancia final de 0.057 m en 267 pasos.

- CONCLUSIONES

El Double DQN demuestra que una modificación algorítmica mínima: desacoplar selección y evaluación de acciones mediante una segunda red, produce una mejora sustancial en la velocidad de convergencia respecto al DQN estándar. Converte 2 000 episodios antes, alcanza distancias finales medias significativamente mejores y prácticamente elimina las colisiones en la fase final del entrenamiento.

La limitación que persiste es la misma que en el DQN: la arquitectura de la red es idéntica para todos los estados y acciones, sin distinguir entre el valor intrínseco de un estado y la ventaja relativa de cada acción. Esta distinción es precisamente la innovación arquitectónica que introduce el Dueling DQN, el siguiente algoritmo implementado.

5.3) DUELING DEEP Q-NETWORK (DUELING DQN)

El Dueling DQN mantiene el mismo algoritmo de entrenamiento que el DQN estándar: experience replay, política ϵ -greedy y descenso del gradiente sobre el MSE de Bellman e introduce una única innovación, pero de naturaleza distinta a la del Double DQN: en lugar de modificar el target de Bellman, modifica la arquitectura interna de la red neuronal.

La idea central es descomponer $Q(s, a)$ en dos cantidades con significado propio:

- $V(s)$: función de valor del estado. Mide cuánta recompensa acumulada espera el agente desde el estado s , independientemente de la acción que tome.
- $A(s, a)$: función de ventaja. Mide cuánto mejor es la acción a respecto a la media de las acciones disponibles en el estado s .

Ambas se combinan para recuperar los Q-values:

$$Q(s, a) = V(s) + (A(s, a) - 1/|A| \cdot \sum_a A(s, a'))$$

La resta de la media de ventajas en lugar del máximo estabiliza el entrenamiento evitando que V y A sean ambiguas: si se restara el máximo, A siempre sería ≤ 0 y la red tendría incentivo a colapsar A a cero en la acción óptima, perdiendo la descomposición.

Arquitectónicamente, la red comparte un tronco de dos capas densas de 64 neuronas con ReLU y se bifurca en dos cabezas independientes al final:

- Cabeza de valor: Linear(64 $\rightarrow$ 1), produce un escalar $V(s)$.
- Cabeza de ventaja: Linear(64 $\rightarrow$ 7), produce un valor $A(s, a)$ por cada acción.

Esta separación es especialmente útil en el entorno de aparcamiento porque existen estados por ejemplo, avanzar por el pasillo central sin obstáculos próximos donde el valor del estado es alto independientemente de si se gira a izquierda o derecha. La red puede aprender $V(s)$ con precisión sin necesidad de visitar cada par (s, a) individualmente, lo que acelera la convergencia de forma notable.

HIPERPARÁMETROS

Hiperparámetros	Valor	Justificación
Episodios	2 000	La convergencia se produce en torno al episodio 700-800. Con 2 000 episodios el agente alcanza una tasa de éxito del 58.8 % en los últimos 500, sin haber agotado su margen de mejora. Con más tiempo de cómputo los resultados serían sensiblemente superiores.
Pasos máximos / episodio	300	Igual que los anteriores.
Warmup steps	1 000	Reducido respecto al DQN y Double DQN. La arquitectura Dueling generaliza mejor desde el principio, por lo que no necesita un buffer tan cargado antes de empezar a aprender.
Factor de descuento γ	0.95	Ligeramente inferior al de los otros algoritmos. En las primeras pruebas, $\gamma = 0.99$ producía inestabilidad en la fase de exploración de este entorno con la arquitectura Dueling; reducirlo a 0.95 estabilizó el aprendizaje sin penalizar la planificación a medio plazo.
Tasa de aprendizaje lr	1×10^{-3}	Más alta que en el DQN y Double DQN (5×10^{-4}). La arquitectura Dueling, al descomponer Q en V y A , produce gradientes más informativos que permiten dar pasos de actualización más grandes sin desestabilizar el entrenamiento.
Tamaño de batch	64	Reducido respecto a los anteriores (128). Batches más pequeños introducen más ruido en el gradiente, lo que actúa como regularización implícita y favorece la exploración del espacio de parámetros en las primeras etapas.
Tamaño replay buffer	10 000	Más pequeño que en los anteriores (60 000). Con una arquitectura que generaliza mejor, no es necesario almacenar tantas transiciones pasadas; un buffer más compacto mantiene experiencias más recientes y relevantes.

ϵ inicial	1.0	Igual que los anteriores.
ϵ mínimo	0.05	Más bajo que en el DQN y Double DQN (0.1). Con una política que converge más rápido, se puede reducir la exploración residual sin riesgo de quedar atrapado en óptimos locales.
ϵ decay	0.995	Más rápido que en los anteriores. Con solo 2 000 episodios y una convergencia temprana, conviene pasar a explotación antes.
Frecuencia de aprendizaje	Cada 4 pasos	Igual que los anteriores.
Arquitectura	18 $\rightarrow$ 64 $\rightarrow$ 64 $\rightarrow$ {V: 1, A: 7}	Tronco compartido de dos capas de 64 neuronas + bifurcación en cabeza de valor (escalar) y cabeza de ventaja (7 valores). Más eficiente que el Double DQN (128 neuronas) porque la descomposición V+A hace que cada parámetro aporte información más específica.

- RESULTADOS

El Dueling DQN es, con diferencia, el algoritmo que converge más rápido de los tres implementados. Ya en el tramo 600-800 el 76 % de los episodios terminan con el vehículo a menos de 0.35 m de la plaza y la tasa de éxito estricta supera el 13 %. En el episodio más o menos 1 050 la media móvil de 50 episodios cruza el 50 % de éxito un hito que el DQN no alcanza en 7 000 episodios y el Double DQN no alcanza en 5 000.

La comparativa al mismo horizonte temporal es especialmente reveladora: a los 2 000 episodios, el DQN tiene una tasa de éxito del 0 % y el Double DQN del 0 %, mientras el Dueling DQN ya ha alcanzado el 32.2 % de éxito con una distancia media final de 1.80 m frente a 5.73 m y 4.52 m respectivamente. La arquitectura de dos cabezas le permite aprender qué estados son valiosos mucho antes de aprender a distinguir con precisión entre acciones.

En los últimos 500 episodios del entrenamiento la tasa de éxito media es del 58.8 %, el reward medio de 104.29 y las colisiones prácticamente desaparecen (0.6 %). Estos resultados se obtienen con solo 2 000 episodios, menos de un tercio de los empleados por el DQN y menos de la mitad que el Double DQN.

Es importante señalar que estos 2 000 episodios no representan el techo del algoritmo. El entrenamiento se detuvo por limitaciones de tiempo de cómputo, no por convergencia. La tendencia en los últimos tramos muestra que la política seguía mejorando activamente; con un entrenamiento completo de duración comparable al DQN o al Double DQN, los resultados del Dueling DQN serían sensiblemente superiores.

- CONCLUSIONES

El Dueling DQN es el algoritmo más eficiente de los tres implementados. Su innovación arquitectónica, descomponer $Q(s, a)$ en $V(s)$ y $A(s, a)$, le permite aprender el valor de los estados de forma independiente a las acciones, lo que acelera drásticamente la convergencia en entornos donde muchas acciones tienen efectos similares desde ciertos estados, como ocurre en el pasillo de acceso a la plaza de aparcamiento.

Con solo 2 000 episodios alcanza una tasa de éxito del 58.8 %, superando ampliamente los resultados del DQN (0 % de éxito a los 2 000 episodios) y del Double DQN (0 % a los 2 000 episodios). Esta diferencia no se debe a mejores hiperparámetros sino a que la arquitectura extrae información más rica de cada transición, haciendo que cada episodio de entrenamiento aporte más al aprendizaje.

La principal limitación de los resultados presentados no es algorítmica sino práctica: el entrenamiento se interrumpió antes de que la política terminara de converger. Dado el ritmo de mejora observado, se estima que con 5 000-6 000 episodios el Dueling DQN alcanzaría tasas de éxito muy superiores al 80 %, consolidándose como la solución más robusta para este entorno. Como tal, no sería una buena práctica comparar los algoritmos tabulares con los de Deep-learning ya que son dos enfoques totalmente distintos aunque como podemos observar para esta práctica concreta cuadra mejor el enfoque de Deep-Learning por el problema que se está tratando.

6) CONCLUSIONES Y TRABAJO FUTURO:

En este trabajo, los resultados obtenidos muestran que los agentes han sido capaces de aprender políticas funcionales para resolver la tarea.

En el caso de los métodos tabulares, tanto Q-Learning como SARSA consiguen aprender comportamientos válidos de aparcamiento a partir de una discretización del estado. Q-Learning muestra una maniobra final más limpia una vez alcanza la zona de entrada a la plaza, aunque su trayectoria inicial no siempre es la más directa. SARSA, por su parte, tiende a dirigirse de forma más directa hacia la plaza, pero realiza más correcciones cerca de la zona de aparcamiento, especialmente movimientos de avance y retroceso antes de frenar correctamente. Esto refleja diferencias coherentes con la naturaleza de ambos algoritmos: Q-Learning es off-policy y actualiza suponiendo la mejor acción futura, mientras que SARSA es on-policy y aprende teniendo en cuenta la acción realmente tomada durante la exploración.

Sin embargo, no puede afirmarse que las políticas aprendidas por los métodos tabulares sean óptimas. El entorno es continuo, pero Q-Learning y SARSA requieren discretizar las observaciones para construir una tabla Q. Esta discretización implica pérdida de información y limita la precisión de la política final. Por este motivo, aparecen trayectorias no completamente suaves, movimientos de corrección y aparcamientos que no siempre terminan exactamente en el centro de la plaza. Aun así, estos métodos han sido útiles para obtener una primera solución interpretable.

Los métodos basados en Deep Learning permiten superar parcialmente esta limitación al trabajar directamente con el vector de observación continuo. El DQN estándar demuestra que una red neuronal puede aproximar la función $Q(s,a)$ sin necesidad de una tabla explícita. No obstante, su convergencia es más lenta y presenta dificultades para cumplir simultáneamente todas las condiciones estrictas de aparcamiento. Esto se debe a la sobreestimación de los valores Q propia del DQN estándar, donde la misma red selecciona y evalúa las acciones.

El Double DQN mejora esta situación al desacoplar la selección y la evaluación de acciones mediante una red online y una red target. Esta modificación reduce la sobreestimación y produce una convergencia más estable. Los resultados muestran que Double DQN aprende a situar el vehículo cerca de la plaza con mayor rapidez que el DQN estándar.

El Dueling DQN ha sido el enfoque más prometedor dentro de los métodos con redes neuronales. Al separar el valor del estado y la ventaja de cada acción, permite aprender de forma más eficiente qué estados son buenos independientemente de la acción concreta. Esta característica resulta especialmente útil en el entorno de aparcamiento, donde existen zonas del mapa en las que varias acciones pueden tener efectos similares. Con menos episodios que los anteriores alcanza una tasa de éxito superior y muestra una convergencia más rápida. Aun así, su entrenamiento se detuvo por limitaciones de tiempo, por lo que probablemente no se alcanzó su máximo rendimiento.

Los resultados indican que el problema planteado se puede resolver mediante Aprendizaje por Refuerzo, pero también muestran que el rendimiento depende mucho de la representación del estado, la función de recompensa y el equilibrio entre exploración y explotación. En las primeras versiones del entorno, el agente podía aprender comportamientos no deseados, como quedarse oscilando entre acciones opuestas o detenerse en posiciones poco útiles. Para corregirlo fue necesario refinar la recompensa, añadiendo penalizaciones por falta de progreso, proximidad excesiva a obstáculos y oscilación. Esto confirma que el diseño de la recompensa es una parte crítica del modelado en Aprendizaje por Refuerzo.

Como principales limitaciones contamos con:

- Discretización del estado en métodos tabulares (limita la precisión de la política y puede producir movimientos poco suaves.)
- Condición de éxito estricta (debe cumplir simultáneamente varios criterios y esto dificulta el aprendizaje, especialmente en los algoritmos con redes neuronales, donde el agente puede aprender a llegar a la plaza pero no siempre frenar en el instante exacto.)
- Tiempo de entrenamiento (Deep Reinforcement Learning requieren muchos episodios y muchas actualizaciones de red. En particular, Dueling DQN seguía mostrando tendencia de mejora cuando se detuvo el entrenamiento)
- Entorno determinista y simplificado (no incluye ruido en sensores, incertidumbre en la ejecución de acciones ni una geometría realista del vehículo. Esto facilita el entrenamiento, pero limita la transferencia directa a un escenario real.
- Posición inicial fija (permite estudiar bien la maniobra definida, pero reduce la generalización)

Como trabajo futuro pensamos que lo primordial de estas limitaciones es introducir variabilidad en el entorno, aleatorizando la posición inicial del vehículo o la localización de los obstáculos. En un principio pensamos en vincular este proyecto con la asignatura de Robótica ya que tiene bastante que ver e incluir sensores con el uso de ROS2, podría ser otra mejora a implementar que sería de ayuda para introducir ruido. Esto permitiría comprobar si el agente aprende una política general de aparcamiento o si simplemente se adapta a una configuración concreta del mapa. En escenarios más realistas, también sería útil representar el vehículo con dimensiones físicas en lugar de como un punto, ya que las colisiones y maniobras de entrada a la plaza serían más precisas. Queremos dejar claro que esto simplemente ha sido una aproximación para introducirnos en el mundo de la conducción autónoma y que somos conscientes de que es una aproximación bastante preliminar, pero nos ha servido para conocer con que algoritmos poder enfrentarlo y que ajustes hacer para ello.

En conclusión, el trabajo demuestra que el Aprendizaje por Refuerzo es una herramienta adecuada para abordar una tarea secuencial de aparcamiento autónomo en un entorno simulado. La política aprendida no es óptima en sentido estricto, pero sí funcional, segura en términos de colisiones y suficientemente buena para validar el modelado propuesto.